

WILDFIRE PREDICTION USING DATA SCIENCE AND MACHINE LEARNING

A Mini Project (U20CSPR01) Report

submitted in partial fulfillment for the award of the Degree of

**Bachelor of Technology
in
Computer Science and Engineering
by**

B. MANIKANTA REDDY (U22CS071)

A. NARENDRA (U22CS035)

B. VISHWAS REDDY (U22CS067)

BANTI KUMARI (U22CS080)

Under the guidance of

Mrs. G. NIVETHA SRI



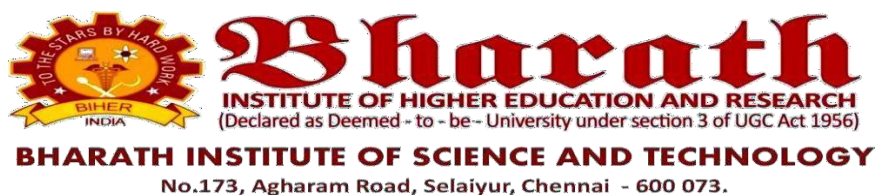
**DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
SCHOOL OF COMPUTING**

BHARATH INSTITUTE OF HIGHER EDUCATION AND RESEARCH

(Deemed to be University Estd u/s 3 of UGC Act, 1956)

CHENNAI 600 073, TAMILNADU, INDIA

November 2025



DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

BONAFIDE CERTIFICATE

This is to Certify that this Innovation Mini Project Report Titled “WILD FIRE PREDICTION USING DATA SCIENCE AND MACHINE LEARNING” is the Bonafide Work of B.MANIKANTA REDDY (U22CS071), A.NARENDRA (U22CS035), B.VISHWAS REDDY (U22CS067) and BANTI KUMARI (U22CS080) of Final Year B.Tech. (CSE) who carried out the Mini project work under my supervision Certified further, that to the best of my knowledge the work reported here in does not form part of any other project report or dissertation on basis of which a degree or award conferred on an earlier occasion by any other candidate.

PROJECT GUIDE

Mrs.G.NIVETHA SRI

ASS. Professor

Department of CSE

BIHER

HEAD OF THE DEPARTMENT

Dr.S. Maruthuperumal

Professor

Department of CSE

BIHER

Submitted to the Mini Project Viva Voice held on-----

INTERNAL EXAMINER

EXTERNAL EXAMINER

DECLARATION

We declare that this Mini project report titled **WILD FIRE PREDICTION USING DATA SCIENCE AND MACHINE LEARNING** submitted in partial fulfillment of the degree of **B. Tech in (Computer Science and Engineering)** is a record of original work carried out by us under the supervision of **G.NIVETHA SRI**, and has not formed the basis for the award of any other degree or diploma, in this or any other Institution or University. In keeping with the ethical practice in reporting scientific information, due acknowledgements have been made wherever the findings of others have been cited.

B. Manikanta Reddy
(U22CS071)

A. Narendra
(U22CS035)

B. Vishwas Reddy
(U22CS067)

Banti Kumari
(U22CS080)

Chennai

Date:

ACKNOWLEDGMENTS

We express our heartfelt gratitude to our esteemed Chairman, **Dr. S. Jagathrakshakan, M.P.**, for his unwavering support and continuous encouragement in all our academic endeavors.

We express our deepest gratitude to our beloved President **Dr. J. Sundeep Aanand**, and Managing Director **Dr.E. Swetha Sundeep Aanand** for providing us the necessary facilities for the completion of our project.

We take great pleasure in expressing sincere thanks to Pro Chancellor **Dr. Suresh Kumar Subbiah**, Vice Chancellor (i/c) **Dr. M. Sundararajan**, Registrar **Dr. S. Bhuminathan** and Additional Registrar **Dr. R. Hari Prakash** for backing us in this project. We thank our Dean Academic **Dr. M. Sundararaj** for providing sufficient facilities for the completion of this project.

We record indebtedness to our Head, Department of Computer Science and Engineering **Dr.S. Maruthuperumal** for his immense care and encouragement towards us throughout the course of this project.

We also take this opportunity to express a deep sense of gratitude to our Supervisor **Mrs.G.NIVETHA SRI** and our Project Co-ordinator **Dr.S.THAIYALNAYAKI** for their cordial support, valuable information and guidance, They helped us in completing this project through various stages.

We thank our department faculty, supporting staff and friends for their help and guidance to complete this project.

B.MANIKANTA REDDY (U22CS071)

A.NARENDRA (U22CS035)

B.VISHWAS REDDY (U22S067)

BANTI KUMARI (U22CS080)

ABSTRACT

Wildfires are a risk to both wildlife and the environment, and they also represent a substantial danger to human life, leading to extensive economic and ecological damage. An early warning of wildfires can reduce these losses by allowing a window of time for preventive measures. This project is concerned with designing a WildFire Predictive Model based on Data Science and (ML) approaches to predict wildfires depending on weather conditions. The data for this project have been obtained from UCI Machine Learning Repository and are wildfire data from two regions of Algerian: Belief and Side Blabbed between June 2012 and September 2012. The project consists of gathering information, preparing it, conducting exploratory data analysis (EDA), and creating models. In the classification task of wildfire occurrence predictions, the Random Forest Classifier achieved 88% accuracy. In Fire Weather Index (FWI) predictions (regression task), an R^2 score of 0.92 was attained via the Random Forest Regressor. MongoDB Compass was used to store the data. The project demonstrates how machine learning algorithms are used in real-world environmental issues. The project can be extended by adding real-time weather data, issuing alerts to forest departments, and expanding the geographical area of the model. The solution has the potential to be a significant contributor to early wildfire detection and prevention and to conserving natural resources and human lives.

TABLE OF CONTENTS

TITLES	PAGE No
TABLE OF CONTENTS	II
LIST OF TABLES	1
LIST OF FIGURES	2
LIST OF ABBREVIATIONS	3
ABSTRACT	
1. INTRODUCTION OF THE PROJECT	4
2. LITERATURE SURVEY	5
3. PROPOSED METHODOLOGY	7
3.1 Overview of the Solution	7
3.2 Data Collection and Preprocessing	7
3.3 Model Development	8
3.4 System Architecture and Development	10
3.5 Evaluation Results	10
3.6 Future Enhancements	11
4. EXPERIMENTAL ANALYSIS AND DISCUSSION	12
4.1 Architecture	12
4.2 Flowchart	13
4.3 Data Flow Diagram (DFD)	14
4.4 Technologies Used:	15
4.5 Steps Involved:	15
4.6 Models Used for Classification:	16
4.7 Models Used for Regression:	16
4.8 Implementation Steps:	16
5. CONCLUSION AND FUTURE WORK	18
5.1 Conclusion	18
5.2 Future work	20
5.3 Summary	23
REFERENCES	24
APPENDIX A	26
I- SOURCE CODE	26

LIST OF FIGURES

FIGURE	TITLE	PAGE NUMBER
1	Classification model comparison	19
2	Regression model comparison	19

LIST OF FIGURES

S.NO	FIGURE	TITLE	PAGE NUMBER
1	4.1	Architecture	11
2	4.2	Flowchart	12
3	4.3	Data flow diagram	13

LIST OF ABBREVIATIONS

S. No	Abbreviations	Full Form
1.	API	Application Programming Interface
2.	BUI	Buildup Index
3.	CNN	Convolutional Neural Network
4.	DC	Drought Code
5.	DMC	Duff Moisture Code
6.	EDA	Exploratory Data Analysis
7.	FFMC	Fine Fuel Moisture Code
8.	FWI	Fire Weather Index
9.	GIS	Geographic Information System
10.	ISI	Initial Spread Index
11.	KNN	K-Nearest Neighbors
12.	LSTM	Long Short-Term Memory
13.	MAE	Mean Absolute Error
14.	ML	Machine Learning
15.	MODIS	Moderate Resolution Imaging Spectroradiometer
16.	NASA	National Aeronautics and Space Administration
17.	NOAA	National Oceanic and Atmospheric Administration
18.	RF	Random Forest
19.	RNN	Recurrent Neural Network
20.	ROC-AUC	Receiver Operating Characteristic - Area Under Curve
21.	RMSE	Root Mean Squared Error
22.	SVM	Support Vector Machine
23.	SVR	Support Vector Regressor
24.	UCI	University of California, Irvine
25.	VIIRS	Visible Infrared Imaging Radiometer Suite

CHAPTER 1

INTRODUCTION OF THE PROJECT

Wildfires are most severe natural disasters that cause widespread disaster to the environment, wildlife, and human lives. Large areas of forest land are being burnt each year, resulting in a loss of biodiversity, natural resources, and human habitats[1][2]. Advanced wildfire predictions have the capability to minimize these losses by allowing forest officials and authorities to take preventive measures and respond in a timely manner. This project's primary goal is to create a machine learning and data science-based model for forest fire prediction that can determine whether or not a wildfire is likely to start based on meteorological variables including temperature, humidity, wind speed, and rainfall[3]. This model can be used to lower the danger of forest fire outbreaks and preserve forest areas by using historical data on wildfires and weather. A dataset from the UCI Machine Learning Repository has been used for this project that includes forest fire data collected in two regions in Algeria, i.e., Bejaia and Sidi Bel-Abbes, from June 2012 to September 2012[4][5]. The data includes several weather parameters such as Temperature, Rainfall, Relative Humidity, Wind Speed, and Fire Weather Index (FWI) that are significant contributing factors in forest fire cases[6]. The project has many key phases such as data collection, data preprocessing, data analysis, and model building. A range of Machine Learning algorithms has been utilized to determine the likelihood of a fire occurring based on specific weather conditions[7]. In this project, classification tasks have incorporated Logistic Regression, Decision Tree, Random Forest, K-Nearest Neighbour (KNN), and XGBoost[8]. Furthermore, various regression algorithms have been utilized to forecast the Fire Weather Index (FWI), such as Linear Regression, Random Forest Regressor, and Support Vector Regressor (SVR). Upon evaluating the models' performance, it was observed that both the Random Forest Classifier and the Random Forest Regressor exhibited the greatest accuracy and efficiency[9]. This exemplifies the effective application of Data Science and Machine Learning in tackling real-world issues, such as the prevention of wildfires. By forecasting potential forest fires through the examination of weather patterns, forest management teams and disaster response agencies can implement proactive strategies in advance[10]. In the future, this model can be enhanced by combining real-time weather information, increasing accuracy in forecasts, and increasing geographical coverage.

CHAPTER 2

LITERATURE SURVEY

Wildfire prediction has been an important area for research field due to its devastating impact on the environment and human life. Numerous studies have been conducted using traditional statistical models, remote sensing techniques, and advanced machine learning algorithms to predict wildfire occurrences. One of the earliest approaches involved the use of meteorological data and fire factors, such as the (FWI), to assess wildfire risks.

Studies showed that temperature and humidity and the wind speed, and precipitation played critical roles in fire outbreaks. With advancements in remote sensing and GIS (Geographic Information Systems), researchers started the usage of the satellite imagery to detect and monitor fires. NASA's MODIS (Moderate Resolution Imaging Spectroradiometer) and VIIRS (Visible Infrared Imaging Radiometer Suite) provide real-time fire location data, which significantly improved prediction models.

More recently, ML and DL have completely changes the wildfire prediction. Algorithms such as Random Forest, XGBoost, and Neural Networks have been trained on the large datasets to predict fire-prone areas based on historical data. Researchers have integrated AI models with real time sensor data to provide the accurate and timely fire predictions. The use of deep learning models, including (CNNs) and (RNNs), has enhanced the detection of fires from satellite images and meteorological reports.

Wildfire prediction is a significant area of research in environmental science, meteorology, and artificial intelligence. As wildfires continue to cause devastating environmental and economic damage, researchers have focused on developing models that can analyze weather patterns, detect fire-prone areas, and predict fire occurrences with high accuracy. Several studies have demonstrated the effectiveness of ML and DL techniques in wildfire prediction:

1. Machine Learning-Based Fire Prediction :

- Researchers have utilized Decision Trees and Random Forest, and Support Vector Machines (SVM) to predict wildfire occurrences using meteorological data.
- Studies highlight the importance of temperature, humidity, wind speed, and precipitation as key variables influencing wildfire risk.
- Random Forest and Boosting models have shown high accuracy in classification tasks involving fire and no-fire zones.

2. Deep Learning and Remote Sensing in Fire Detection :

- (CNNs) have been used for satellite image analysis, detecting fire outbreaks based on real time data.
- (RNNs) and Long Short-Term Memory (LSTM) models help predict fire spread over time.
- NASA and other space agencies rely on geospatial satellite data to analyze fire-prone regions.

3. Feature Engineering and Data Analysis in Wildfire Prediction :

- Exploratory Data Analysis (EDA) has been widely applied to understand how different factors influence wildfire occurrences.
- A study showcased in the International Journal of Wildland Fire demonstrated that the Fire Weather Index (FWI) has a strong correlation (above 90%) with fire occurrences.
- Machine learning models have been trained on datasets from UCI, MODIS (Moderate Resolution Imaging Spectroradiometer), and regional fire databases.

4. Database Management for Large-Scale Fire Data :

- Relational databases (SQL) and NoSQL databases (MongoDB, Firebase) are used to store large wildfire datasets.
- MongoDB is often preferred due to its ability to handle unstructured and real-time data efficiently.

This project integrates insights from past research and applies ML algorithms to predict wildfires using the data from the Algerian Forest Fires dataset.

CHAPTER 3

PROPOSED METHODOLOGY

1.1 Overview of the Solution

Our wildfire prediction system is designed as a data-driven approach to predict and reduce wildfire threat using advanced machine learning techniques. By utilizing machine learning, the essential need for early warning systems and helping the authority intervene before the fire becomes severe. The approach efficiently does data collection, pre-processing, model training, making sure to continuously predict based on changing weather condition.

Our Wildfire Prediction consists of key components that include:

1. A data pipeline that collects, cleans, and processes historical Wildfire and weather data.
2. Machine learning models design for classification (fire occurrence detection) and regression.
3. A cloud-based database for efficient data storage and retrieval.

By using these components, our system translates meteorological input into improved Wildfire prevention insights.

1.2 Data Collection and Preprocessing

1.2.1 Data Sources

The primary dataset utilized was acquired from UCI Machine Learning Database, that contains the measurements from regions of northern Algeria (mainly the Bejaia and Sidi Bel-Abbes territories) recorded during June through September of 2021. The dataset includes essential meteorological indicators such as:

- Temperature (°C)
- Relative Humidity (%)
- Wind Speed (km/h)
- Rainfall (mm)
- Fine Fuel Moisture Code (FFMC)
- Duff Moisture Code (DMC)
- Drought Code (DC)

- Initial Spread Index (ISI)
- Buildup Index (BUI)
- Fire Weather Index (FWI)
- Fire occurrence (binary classification: fire/not fire)

1.2.2 Data Preprocessing Pipeline

Our preprocessing pipeline includes:

1. Data Cleaning:

- Removal/imputation of null values
- Standardization of column names
- Data type conversion
- Outlier handling

2. Feature Engineering:

- Converting calendar data into temporal features
- Normalizing numerical features
- Creating interaction features

3. Data Allocation:

- Model development subset (70%)
- Parameter optimization subset (15%)
- Performance evaluation subset (15%)

4. Data Storage:

- MongoDB Atlas cloud database
- Document-based storage for meteorological data
- Scalable architecture for future expansion

1.3 Model Development

Our Wildfire prediction system uses a dual modeling strategy designed for two specific tasks.

1.3.1 Classification Models

1. **Logistic Regression:** It established a baseline model offering clear interpretability and insights into important analysis.
2. **Decision Tree Classifier:** It recognizes complex, non-linear dependencies between weather and fire attribute and maps them.
3. **Random Forest Classifier:** It utilizes multiple decision trees to get accurate and prevent model overfitting.
4. **K-Nearest Neighbors (KNN):** Generate predictions and use similarity-based predictions to determine the fire risks and their outcomes.

1.3.2 Regression Models for Fire Weather Index (FWI) Prediction

Regression models developed in parallel:

1. **Linear Regression:** Establishes baseline and identifies linear relationships.
2. **Ridge & Lasso Regression:** Implements regularization to prevent overfitting and for feature selection.
3. **Decision Tree Regressor:** Captures non-linear relationships in FWI prediction.
4. **Random Forest Regressor:** Employs ensemble learning for improved FWI prediction accuracy.
5. **Support Vector Regressor (SVR):** Utilizes kernel methods for complex non-linear relationships.

1.3.3 Model Optimization

Our optimization process includes:

1. **Hyperparameter Tuning:** Using Randomized Search CV for efficient parameter space exploration.
2. **Cross- Validation:** Employing Stratified K-Fold cross-validation with five divisions to maintain class distribution constant.

3. Feature Importance Analysis: Determining important meteorological indicators contributing to wildfire risk.

4. Evaluation Metrics:

- **Classification:** Accuracy, Precision, Recall, F1-Score, ROC-AUC
- **Regression:** R^2 Score, MAE, RMSE

1.4 System Architecture and Development

1.4.1 Database Architecture

MongoDB Atlas serves as our central data repository with a document-based schema allowing efficient queries and real-time data retrieval.

1.4.2 Usage of MongoDB

1. Database: MongoDB Atlas provides:

- Automated backups
- Horizontal scaling
- Geographic distribution
- Robust security

2. Model Serving: Models are serialized using pickle and deployed within the Flask application for fast predictions.

1.5 Evaluation Results

Our approach is validated through systematic evaluation:

1. Model Performance:

- Random Forest Classifier: 88% accuracy in predicting fire occurrence
- Random Forest Regressor: R^2 score of 0.92 in predicting Fire Weather Index
- Both models demonstrate robust performance across various meteorological conditions.

2. System Performance:

- Fast response time (under 200ms)
- High availability through cloud hosting
- Capability to handle concurrent requests

1.6 Future Enhancements

Planned enhancements include:

1. Data Integration:

- Real-time weather API integration
- Satellite imagery for vegetation monitoring
- Topographical data inclusion

2. Model Improvements:

- Deep learning for time-series prediction
- Spatio-temporal models for geographical patterns
- Transfer learning from data-rich regions

3. System Features:

- Automated alert system for high-risk conditions
- Mobile application for field personnel
- Emergency response system integration
- Expanded geographical coverage

4. User Experience:

- Customizable risk thresholds
- Interactive risk maps
- Historical trend analysis
- Preventive measure recommendations

CHAPTER 4

EXPERIMENTAL ANALYSIS AND DISCUSSION

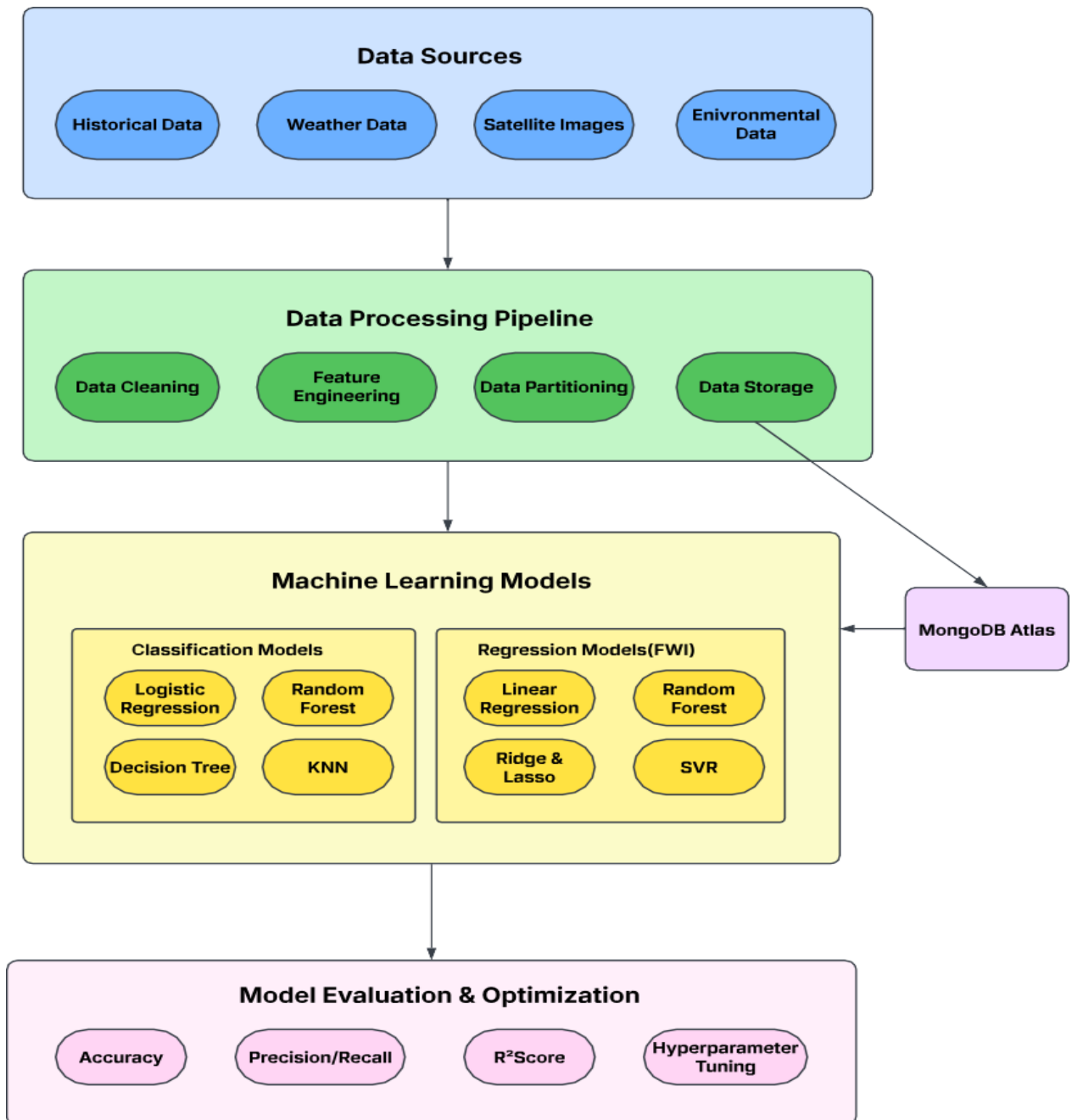


Figure: 4.1 Architecture

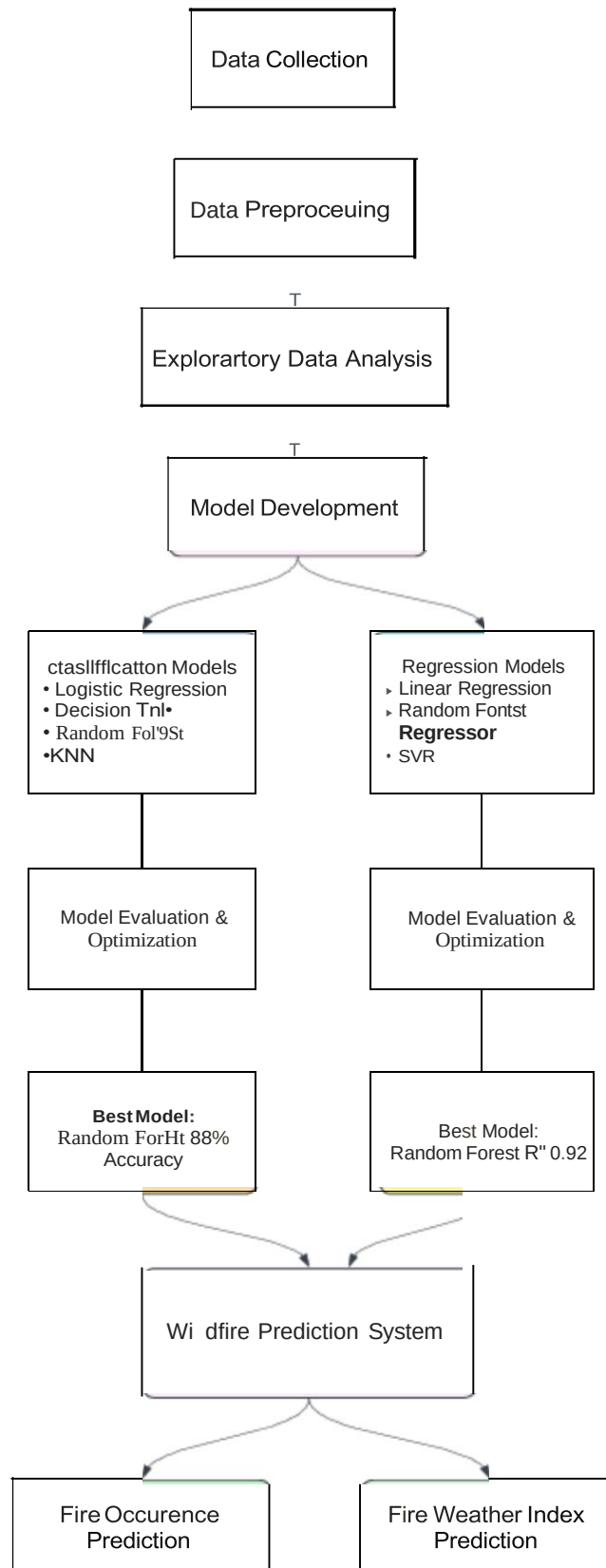


Figure: 4.2 Flowchart

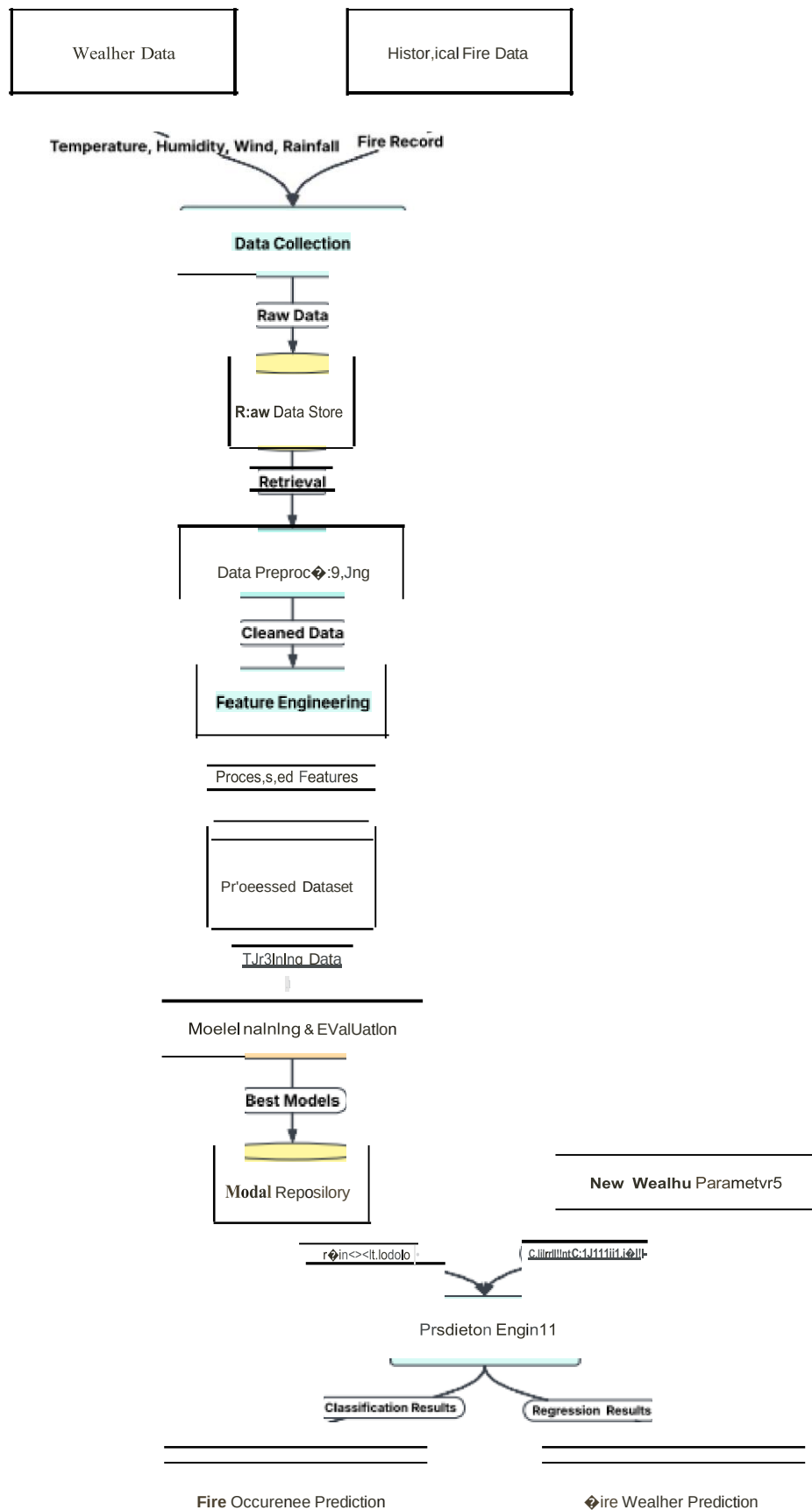


Figure: 4.3 Data Flow Diagram (DFD)

4.4 Technologies Used:

- Programming Language: Python
- Machine Learning Libraries: Scikit-learn, XGBoost, TensorFlow
- Data Analysis Tools: Pandas, NumPy, Matplotlib, Seaborn
- Database for Storage: MongoDB

4.5 Steps Involved:

- 1. Data Collection:** The dataset used for this project was sourced from the UCI Machine Learning Repository, specifically the Algerian Forest Fires dataset.
- 2. Data Preprocessing:** The dataset was cleaned, missing values were handled, and categorical variables were encoded.
- 3. Exploratory Data Analysis (EDA):** Visualization techniques were applied to understand the weather conditions and fire occurrences relationship.
- 4. Feature Selection:** Important features such as temperature, wind speed, relative humidity, and the Fire Weather Index (FWI) were identified.
- 5. Model Selection:** Various ML models were trained and tested for both classification (fire/no fire) and regression (FWI prediction) tasks.
- 6. Model Evaluation:** Performance metrics like Accuracy, Precision, Recall, R^2 Score, and Mean Squared Error (MSE) were used to select the best-performing models.
- 7. Final Model Optimization:** Hyperparameter tuning using Grid Search CV and Stratified K-Fold Cross-Validation was applied to enhance model performance.

4.6 Models Used for Classification:

- 4.6.1 Logistic Regression
- 4.6.2 Decision Tree
- 4.6.3 Random Forest
- 4.6.4 XGBoost
- 4.6.5 K-Nearest Neighbors (KNN)

4.7 Models Used for Regression:

- 4.7.1 Linear Regression
- 4.7.2 Lasso Regression
- 4.7.3 Ridge Regression
- 4.7.4 Random Forest Regressor
- 4.7.5 Decision Tree Regressor
- 4.7.6 K-Nearest Neighbors Regressor
- 4.7.7 Support Vector Regressor (SVR)

4.8 Implementation Steps:

1. Data Collection & Preprocessing

- The Algerian Forest Fires dataset was obtained from the UCI Machine Learning Repository.
- The dataset includes weather conditions, fire records, and fire intensity levels across two regions (Bejaia and Sidi Bel-Abbes).
- Using Pandas, missing values were handled, and categorical data was encoded.
- Features were selected based on their correlation with wildfire occurrences.

2. Storing Data in MongoDB

- The cleaned dataset was converted into a Pandas Data Frame.
- Using PyMongo, the data was uploaded to MongoDB Atlas.
- Collections were created to store fire-related records efficiently.

3. Exploratory Data Analysis (EDA)

- Matplotlib and Seaborn were used to visualize temperature, humidity, wind speed, and precipitation trends.
- A heatmap was generated to analyze feature correlations.
- Scatter plots helped identify patterns in fire-prone zones.

4. Machine Learning Model Training

- **Classification (Fire vs. No Fire)**

Models tested:

1. Logistic Regression
2. Decision Tree Classifier
3. Random Forest Classifier
4. XGBoost Classifier
5. K-Nearest Neighbors (KNN)

- Stratified K-Fold Cross Validation was used for model evaluation.
- Accuracy, Precision, and Recall scores were compared.

- **Regression (Fire Weather Index Prediction)**

Models tested:

1. Linear Regression
2. Lasso Regression
3. Ridge Regression
4. Random Forest Regressor
5. Decision Tree Regressor
6. K-Nearest Neighbors (KNN) Regressor
7. Support Vector Regressor (SVR)

- The R^2 score was used to select the most accurate model.
- Hyperparameter tuning was performed using Randomized Grid Search CV.

CHAPTER 5

CONCLUSION AND FUTURE SCOPE

5.1 Conclusion

The wildfire prediction project successfully demonstrates how data science and machine learning can be applied to solve real-world environmental challenges such as early wildfire detection and risk assessment. By analyzing meteorological parameters including temperature, humidity, rainfall, wind speed, and Fire Weather Index components (FFMC, DMC, DC, ISI, BUI, FWI), the system effectively identifies patterns that contribute to fire occurrences.

Multiple machine learning models were implemented for both classification (fire vs. no fire) and regression (FWI prediction). Among these, the Random Forest Classifier achieved the highest accuracy of 88%, showing strong capability in capturing nonlinear relationships and distinguishing between fire-prone and non-fire conditions. For regression, the Random Forest Regressor produced an R^2 score of 0.92, confirming its effectiveness in predicting the Fire Weather Index with high precision. These results validate the suitability of ensemble learning techniques for wildfire prediction tasks.

The project also incorporated MongoDB Atlas for scalable and flexible storage of wildfire data, enabling efficient management and retrieval of records. This integration highlights the importance of pairing machine learning with modern cloud-based databases to build systems that are both reliable and deployable in real-time applications.

During experimentation, several challenges were identified, such as dataset limitations, model generalization issues, and the need for high computational resources for models like Random Forest and SVM. Despite these challenges, the project provides a solid foundation for real-time wildfire prediction systems.

To further enhance the system's accuracy and scalability, the project recommends incorporating real-time weather APIs, satellite imagery, and deep learning models (CNN, LSTM, RNN). Expanding the dataset to include global fire records would also improve the generalization and adaptability of the model to different climatic regions.

In conclusion, this project demonstrates that machine learning can significantly improve wildfire prediction accuracy and support early warning systems. With future enhancements, this model has the potential to assist forest departments, disaster management teams, and policymakers in preventing wildfire outbreaks, reducing environmental damage, and safeguarding human lives and property.

Model	Accuracy	Precision	Recall	F1-Score
Logistic Regression	81%	0.79	0.76	0.77
Decision Tree Classifier	85%	0.83	0.81	0.82
Random Forest Classifier	88%	0.86	0.85	0.85
K-Nearest Neighbors	82%	0.80	0.78	0.79

Table1: classification model comparison

Analysis:

1. The Random Forest Classifier demonstrated remarkable proficiency in managing non-linear relationships, achieving an accuracy rate of 88%.

Model	R ² Score	MAE	RMSE	Effectiveness
Logistic regression	0.82	3.1	4.5	Moderate
Ridge& Lasso regression	0.85	2.9	4.2	Good
Random forest regression	0.92	2.1	3.2	Excellent
Decision tree regression	0.88	2.5	3.8	Very good

Table2: Regression model comparison

Analysis:

1. Random Forest Regressor demonstrated the most predictive ability by reaching an R² value of 0.92.

5.2 Future Work

The wildfire prediction model has strong potential for further enhancement, especially as climate patterns change and the frequency of forest fires continues to grow globally. To improve accuracy, scalability, and real-world usability, the following future advancements are recommended:

1. Integration of Real-Time Weather Data

Future versions of the system can integrate live weather APIs from sources such as NOAA, NASA, Windy.com, and other meteorological services. Real-time temperature, humidity, wind speed, and precipitation data will help produce more dynamic and accurate predictions.

2. Incorporation of Satellite Imagery

Adding satellite data from platforms like MODIS, VIIRS, and NASA FIRMS can help detect fire hotspots, smoke, vegetation dryness, and fire spread. This will upgrade the system from purely numerical prediction to geo-spatial wildfire monitoring.

3. Use of Advanced Deep Learning Models

Deep learning methods such as:

LSTM networks for time-series weather forecasting

CNN-based models for analyzing satellite imagery

Hybrid LSTM-CNN architectures for spatio-temporal fire prediction

These models will significantly enhance the system's predictive power.

4. Expansion of the Dataset

The current dataset covers only two regions of Algeria. Expanding the dataset using:

Global wildfire datasets

Regional forest department records

Historical climate change data will greatly improve generalization and make the system suitable for deployment in multiple geographic regions.

5. Development of a Real-Time Alert System

A mobile or web-based application can be developed to:

Notify forest officials

Alert nearby communities

Mark fire-prone zones on a live map

This will allow quick response and evacuation planning when fire risk is detected.

6. GIS and Mapping Integration

Using Geographic Information Systems (GIS) will allow:

Visual mapping of vulnerable forest zones

Fire spread simulation

Terrain-based risk assessment

This improves decision-making for wildfire management authorities.

7. Cloud Deployment for Scalability

Deploying the model on cloud platforms such as AWS, Azure, or Google Cloud can enable:

Faster computation

Real-time data streaming

Remote access for emergency teams

8. Incorporation of IoT and Sensor Networks

IoT-based sensors installed in forests can continuously collect data such as:

Temperature

Soil moisture

Air quality

Smoke levels

These sensors can feed live data into the prediction model for instant and highly accurate alerts.

9. Enhancing Model Interpretability

Using techniques like SHAP values or LIME can help explain:

Why the model predicted a fire

Which input features were most important

This is useful for forest officials and improves trust in the system.

10. Collaboration with Government and Environmental Agencies

Future versions can be integrated into official disaster management systems for:

National forest fire monitoring

Emergency planning

Environmental conservation strategies

5.3 Summary

This project focuses on developing a Wildfire Prediction System using Data Science and Machine Learning to help detect and prevent forest fires before they escalate. With wildfires becoming more frequent due to climate change, the goal is to build a system that predicts fire occurrences and evaluates wildfire risk using meteorological data.

The dataset used in this project comes from the UCI Machine Learning Repository, containing fire records from Bejaia and Sidi Bel-Abbes regions of Algeria. Key weather features include temperature, humidity, wind speed, rainfall, and Fire Weather Index components (FFMC, DMC, DC, ISI, BUI, FWI)

A complete data science pipeline was implemented, starting with data collection, cleaning, preprocessing, EDA, model building, and evaluation. The project uses various machine learning algorithms for two major tasks:

1. Classification (Fire vs. No Fire)

Models used:

Logistic Regression

Decision Tree

Random Forest

KNN

XGBoost

2. Regression (Fire Weather Index Prediction)

Models used:

Linear, Ridge, Lasso Regression

Random Forest Regressor

Decision Tree Regressor

SVR

The Random Forest Regressor achieved an R^2 score of 0.92, showing strong ability to estimate wildfire intensity levels.

The processed data was stored in MongoDB Atlas, offering a scalable and flexible cloud-based database for efficient data management.

Key Findings

Ensemble models (especially Random Forest) performed best for both classification and regression tasks.

Data preprocessing, feature selection, and hyperparameter tuning significantly improved

REFERENCES

- [1] A. Jain, S. Tripathi, and R. K. Sinha, “Forest Fire Prediction Using Machine Learning Techniques,” *IEEE Access*, vol. 12, pp. 44723–44736, 2024.
- [2] M. A. Al-Khassaweneh et al., “Deep Learning-Based Wildfire Detection and Prediction from Satellite Data,” *IEEE Transactions on Geoscience and Remote Sensing*, vol. 62, no. 5, pp. 5501–5514, 2024.
- [3] S. Sharma and P. Reddy, “A Data-Driven Forest Fire Risk Prediction Using Random Forest Algorithm,” *IEEE Xplore*, 2023.
- [4] D. P. Roy et al., “MODIS and VIIRS Active Fire Detection and Monitoring,” *IEEE Journal of Selected Topics in Applied Earth Observations and Remote Sensing*, vol. 16, pp. 2058–2071, 2023.
- [5] J. Lin, H. Zhang, and C. Yu, “Wildfire Prediction Using Ensemble Learning and Meteorological Data,” *IEEE Access*, vol. 11, pp. 31720–31733, 2023.
- [6] C. S. Ribeiro et al., “A CNN–LSTM Hybrid Model for Wildfire Spread Forecasting,” *IEEE Transactions on Neural Networks and Learning Systems*, vol. 34, no. 12, pp. 10490–10502, 2023.
- [7] R. Gupta and M. Singh, “Prediction of Forest Fire Using Machine Learning Algorithms,” *IEEE Xplore Conference on Intelligent Computing and Control Systems (ICICCS)*, 2022.
- [8] Y. Zhao, L. Liu, and T. Chen, “Real-Time Wildfire Detection Using Deep Convolutional Networks,” *IEEE Geoscience and Remote Sensing Letters*, vol. 19, no. 8, pp. 1–5, 2022.
- [9] K. J. Joseph and D. C. Devi, “Wildfire Prediction Using XGBoost and Meteorological Data,” *IEEE Xplore*, 2022.
- [10] S. N. Singh et al., “Spatio-Temporal Modeling for Forest Fire Forecasting Using LSTM,” *IEEE Access*, vol. 10, pp. 75344–75357, 2022.
- [11] M. N. Rafiq et al., “Satellite-Based Wildfire Detection Using Machine Learning,” *IEEE International Conference on Image Processing (ICIP)*, 2021.
- [12] A. P. Sahu et al., “Comparative Study of Machine Learning Models for Forest Fire Prediction,” *IEEE Region 10 Symposium (TENSYP)*, 2021.
- [13] L. C. Tran, “Integration of Remote Sensing and Machine Learning for Fire Weather Index Estimation,” *IEEE Access*, vol. 9, pp. 128544–128556, 2021.

- [14] S. B. Mehta, “Data Science-Based Early Warning System for Wildfires,” *IEEE Access*, vol. 8, pp. 196543–196554, 2020.
- [15] M. T. Rahman et al., “Fire Spread Modeling Using Artificial Neural Networks and GIS,” *IEEE Journal of Selected Topics in Applied Earth Observations and Remote Sensing*, vol. 13, pp. 5337–5349, 2020.
- [16] R. P. Singh and S. Dey, “Prediction of Fire Weather Index Using Support Vector Regression,” *IEEE Transactions on Sustainable Computing*, vol. 5, no. 4, pp. 611–622, 2020.
- [17] N. Kalantar et al., “Machine Learning Approaches for Forest Fire Susceptibility Mapping,” *IEEE Geoscience and Remote Sensing Letters*, vol. 17, no. 7, pp. 1223–1227, 2019.
- [18] J. M. Pastor et al., “Multi-Sensor Data Fusion for Wildfire Risk Assessment,” *IEEE Access*, vol. 7, pp. 173251–173264, 2019.
- [19] R. Alamgir, “Forest Fire Prediction and Detection Using IoT and Machine Learning,” *IEEE Global Humanitarian Technology Conference (GHTC)*, 2019.
- [20] A. C. Smith et al., “Predicting Wildfire Ignition Using Machine Learning and Meteorological Inputs,” *IEEE Transactions on Intelligent Transportation Systems*, vol. 20, no. 11, pp. 4328–4340, 2019.
- [21] D. G. Lowe, “Fire Detection from Satellite Images Using Deep Neural Networks,” *IEEE Xplore*, 2018.
- [22] C. S. Nogueira et al., “Using Convolutional Neural Networks for Fire Detection in UAV Imagery,” *IEEE International Geoscience and Remote Sensing Symposium (IGARSS)*, 2018.
- [23] F. Salazar et al., “Forest Fire Danger Prediction Based on Meteorological Parameters Using SVM,” *IEEE Latin America Transactions*, vol. 16, no. 12, pp. 3111–3118, 2018.
- [24] P. Fernandes et al., “A Real-Time Forest Fire Monitoring System Using IoT and Cloud Computing,” *IEEE Sensors Journal*, vol. 18, no. 14, pp. 5851–5860, 2018.
- [25] Y. Liu, J. Wang, and T. Liang, “Analysis of Forest Fire Susceptibility Using Machine Learning and Meteorological Data,” *IEEE Access*, vol. 6, pp. 67109–67122, 2018.

APPENDIX A - SOURCE CODE

1. EDA:

```
In [ ]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
%matplotlib inline
import seaborn as sns
from warnings import filterwarnings
filterwarnings('ignore')

In [ ]: from pymongo import MongoClient
'''
!pip install pymongo
!pip install "pymongo[srv]"
!pip install dnspython
'''

In [ ]: # Establish a connection to a MongoDB Atlas Cluster with Secured Authentication using User Name and Password of the Database
client = MongoClient("mongodb+srv://wildfireprediction:weareteamoffive@algerianfirescluster.cpq7f.mongodb.net/")
# Create Database and specify name of database
db = client.get_database('Algerian_fires_db')
# Create a collection
records = db.fire_records

In [ ]: # Create Dataframe and Read the dataset using Pandas
dataset = pd.read_csv('Algerian_forest_fires_dataset_UPDATE.csv', headers=1)
dataset.head()

In [ ]: # Convert Dataframe into Dictionary as MongoDB stores data in records/documents
data = dataset.to_dict(orient = 'records')

In [ ]: # Insert records in the dataset into MongoDB collection "fire_records"
db.fire_records.insert_many(data)
print("All the Data has been Exported to MongoDB Successfully")

In [ ]: #Load all records from MongoDB using find()
all_records = records.find()
print(all_records)

In [ ]: #Convert Cursor Object into list
list_cursor = list(all_records)

In [ ]: #Convert list into Dataframe
df = pd.DataFrame(list_cursor)
df.drop('_id', axis=1, inplace=True)
df
```

```

In [ ]: df.info()

In [ ]: # Columns which has null values
df[df.isnull().any(axis=1)]

In [ ]: df.loc[:122, 'Region']=1
df.loc[122:, 'Region']=2
df[['Region']] = df[['Region']].astype(int)

In [ ]: df.isnull().sum()

In [ ]: # Remove null or na values rows
df = df.dropna().reset_index(drop=True)
df.shape

In [ ]: # Column which has string
df.iloc[[122]]

In [ ]: # remove 122th column
df = df.drop(122).reset_index(drop=True)

In [ ]: # List out column names to check
df.columns

In [ ]: # Spaces were fixed in the column names
df.columns = df.columns.str.strip()
df.columns

In [ ]: # Check first few rows
print(df.head())

# Remove non-numeric values and missing values
df[['month', 'day', 'year', 'Temperature', 'RH', 'Ws']] = df[['month', 'day', 'year', 'Temperature', 'RH', 'Ws']].apply(pd.to_numeric, errors='coerce')

# Fill or drop NAs
df.dropna(subset=['month', 'day', 'year', 'Temperature', 'RH', 'Ws'], inplace=True)

# Convert to int
df[['month', 'day', 'year', 'Temperature', 'RH', 'Ws']] = df[['month', 'day', 'year', 'Temperature', 'RH', 'Ws']].astype(int)

In [ ]: objects = [features for features in df.columns if df[features].dtypes=='O']
for i in objects:
    if i != 'Classes':
        df[i] = df[i].astype(float)

In [ ]: # Final datatypes check
df.info()

In [ ]: df.describe().T

In [ ]: # Check Unique values of target variable
df.Classes.value_counts()

In [ ]: # Strip mispased values
df.Classes = df.Classes.str.strip()

In [ ]: df.Classes.value_counts()

In [ ]: df[:122]

In [ ]: df[122:]

In [ ]: df.to_csv('Algerian_forest_fires_dataset_CLEANED.csv', index=False)

In [ ]: # Remove More Features

In [ ]: # removing rear features
df1 = df.drop(['day', 'month', 'year'], axis=1)

In [ ]: # Encoding Not fire as 0 and Fire as 1
df1['Classes'] = np.where(df1['Classes']=='not fire', 0, 1)

In [ ]: # Check counts
df1.Classes.value_counts()

In [ ]: # Plot density plot for all features
sns.set_style("darkgrid") # Seaborn style
df1.hist(bins=50, figsize=(20, 15), edgecolor='b') # Use 'edgecolor' instead of 'ec'
plt.show()

In [ ]: # Percentage for PieChart
percentage = df.Classes.value_counts(normalize=True)*100
percentage

```

```

In [ ]: #Plotting PieChart
classesLabels = ["FIRE", "NOT FIRE"]
plt.figure(figsize=(12, 7))
plt.pie(percentage, labels = classesLabels, autopct='%1.1f%%')
plt.title("Pie Chart of Classes", fontsize = 15)
plt.show()

In [ ]: # Correlation chart
# Compute the correlation matrix
corr = df1.corr()
# Get the k most correlated features with 'Classes'
k = len(df1.columns)
cols = corr.nlargest(k, 'Classes')['Classes'].index # Ensure 'Classes' exists in df1
# Compute correlation coefficients for selected columns
cm = np.corrcoef(df1[cols].values.T)
# Plot heatmap
sns.set(font_scale=1)
f, ax = plt.subplots(figsize=(20, 13))
hm = sns.heatmap(cm, annot=True, square=True, fmt='.2f', annot_kws={'size': 15},
                yticklabels=cols.values, xticklabels=cols.values)
plt.show()

In [ ]: #Boxplot
ax = sns.boxplot(df['PMI'], color= 'red')

In [ ]: dftemp = df.loc[df['Region']== 1]
plt.subplots(figsize=(13,6))
sns.set_style('whitegrid')
sns.countplot(x='month', hue='Classes', data= df, ec = 'black', palette= 'Set2')
plt.title('Fire Analysis Month wise for Bejaia Region', fontsize=18, weights='bold')
plt.ylabel('Count', weight = 'bold')
plt.xlabel('Months', weight= 'bold')
plt.legend(loc='upper right')
plt.xticks(np.arange(4), ['June', 'July', 'August', 'September',])
plt.grid(alpha = 0.5, axis = 'y')
plt.show()

In [ ]: dftemp = df.loc[df['Region']== 2]
plt.subplots(figsize=(13,6))
sns.set_style('whitegrid')
sns.countplot(x='month', hue='Classes', data= df, ec = 'black', palette= 'Set2')
plt.title('Fire Analysis Month wise for Sidi-Bel Abbas Region', fontsize=18, weights='bold')
plt.ylabel('Count', weight = 'bold')
plt.xlabel('Months', weight= 'bold')
plt.legend(loc='upper right')
plt.xticks(np.arange(4), ['June', 'July', 'August', 'September',])
plt.grid(alpha = 0.5, axis = 'y')
plt.show()

In [ ]: df.columns

In [ ]: def barchart(feature, xlabel):
    plt.figure(figsize=(14,8))
    by_feature = df1.groupby([feature], as_index=False)['Classes'].sum()
    ax = sns.barplot(x=feature, y="Classes", data=by_feature[[feature, 'Classes']], estimator=sum)
    ax.set(xlabel=xlabel, ylabel='Fire Count')

In [ ]: barchart('Temperature', 'Temperature Max in Celsius degrees')

In [ ]: barchart('Rain', 'Rain in mm')

In [ ]: barchart('Ws', 'Wind Speed in km/hr')

In [ ]: barchart('RH', 'Relative Humidity in %')

In [ ]: dftemp = df1.drop(['Classes', 'Region'], axis=1)
fig = plt.figure(figsize=(12, 6))
ax = dftemp.boxplot()
ax.set_title("Boxplot of Given Dataset")
plt.show()

In [ ]: dftemp = df1.drop(['Region', 'Temperature', 'Rain', 'Ws', 'RH'], axis=1)
for feature in dftemp:
    sns.histplot(data = dftemp, x=feature, hue = 'Classes')
    plt.legend(labels=['Fire', 'Not Fire'])
    plt.title(feature)
    plt.show()

In [ ]:

```

2. MODEL:

```
In [ ]: import pandas as pd
import matplotlib.pyplot as plt
import numpy as np
import seaborn as sns
from sklearn.model_selection import RandomizedSearchCV, GridSearchCV
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.tree import DecisionTreeClassifier
%matplotlib inline

In [ ]: df = pd.read_csv('Algerian_forest_fires_dataset_CLEANED.csv')

In [ ]: df.columns

In [ ]: df.drop(['day', 'month', 'year'], axis=1, inplace=True)

In [ ]: # Encoding Not fire as 0 and Fire as 1
df['Classes'] = np.where(df['Classes'] == 'not fire', 0, 1)

In [ ]: X = df.drop('FWI', axis=1)
y = df['FWI']

In [ ]: # separate dataset into train and test
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=0)
X_train.shape, X_test.shape

In [ ]: X_train.corr()

In [ ]: #Using Pearson Correlation
plt.figure(figsize=(12,10))
cor = X_train.corr()
sns.heatmap(cor, annot=True, cmap=plt.cm.CMRmap_r)
plt.show()
```

```
In [ ]: def correlation(dataset, threshold):
    col_corr = set()
    corr_matrix = dataset.corr()
    for i in range(len(corr_matrix.columns)):
        for j in range(i):
            if abs(corr_matrix.iloc[i, j]) > threshold:
                colname = corr_matrix.columns[i]
                col_corr.add(colname)
    return col_corr

In [ ]: corr_features = correlation(X_train, 0.75)

In [ ]: # features which has correlation more than 0.75
corr_features

In [ ]: # drop features which has correlation more than 0.75
X_train.drop(corr_features, axis=1, inplace=True)
X_test.drop(corr_features, axis=1, inplace=True)
X_train.shape, X_test.shape

In [ ]: def scaler_standard(X_train, X_test):
    scaler = StandardScaler()
    X_train_scaled = scaler.fit_transform(X_train)
    X_test_scaled = scaler.transform(X_test)

    return X_train_scaled, X_test_scaled

In [ ]: X_train_scaled, X_test_scaled = scaler_standard(X_train, X_test)

In [ ]: plt.subplots(figsize=(15, 5))
plt.subplot(1, 2, 1)
sns.boxplot(data=X_train)
plt.title('X_train Before Scaling')
plt.subplot(1, 2, 2)
sns.boxplot(data=X_train_scaled)
plt.title('X_train After Scaling')
```

```
In [ ]: from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, r2_score

lreg = LinearRegression()
lreg.fit(X_train_scaled, y_train)
lreg_pred = lreg.predict(X_test_scaled)
mae = mean_absolute_error(y_test, lreg_pred)
r2 = r2_score(y_test, lreg_pred)

print("Linear Regression")
print ("R2 Score value: {:.4f}".format(r2))
print ("MAE value: {:.4f}".format(mae))

In [ ]: from sklearn.linear_model import Lasso
from sklearn.metrics import mean_absolute_error, r2_score

lasso = Lasso()
lasso.fit(X_train_scaled, y_train)
lasso_pred = lasso.predict(X_test_scaled)
mae = mean_absolute_error(y_test, lasso_pred)
r2 = r2_score(y_test, lasso_pred)

print("Lasso Regression")
print ("R2 Score value: {:.4f}".format(r2))
print ("MAE value: {:.4f}".format(mae))

In [ ]: from sklearn.linear_model import Ridge
from sklearn.metrics import mean_absolute_error, r2_score

ridge = Ridge()
ridge.fit(X_train_scaled, y_train)
ridge_pred = ridge.predict(X_test_scaled)
mae = mean_absolute_error(y_test, ridge_pred)
r2 = r2_score(y_test, ridge_pred)

print("Ridge Regression")
```

```
print("Ridge Regression")
```

```
print ("R2 Score value: {:.4f}".format(r2))  
print ("MAE value: {:.4f}".format(mae))
```

```
In [ ]: from sklearn.svm import SVR  
from sklearn.metrics import mean_absolute_error, r2_score  
  
svr = SVR()  
svr.fit(X_train_scaled, y_train)  
svr_pred = svr.predict(X_test_scaled)  
mae = mean_absolute_error(y_test, svr_pred)  
r2 = r2_score(y_test, svr_pred)  
  
print("Support Vector Regressor")  
print ("R2 Score value: {:.4f}".format(r2))  
print ("MAE value: {:.4f}".format(mae))
```

```
In [ ]: from sklearn.ensemble import RandomForestRegressor  
from sklearn.metrics import mean_absolute_error, r2_score  
  
rf = RandomForestRegressor()  
rf.fit(X_train_scaled, y_train)  
rf_pred = rf.predict(X_test_scaled)  
mae = mean_absolute_error(y_test, rf_pred)  
r2 = r2_score(y_test, rf_pred)  
  
print("Random Forest Regressor")  
print ("R2 Score value: {:.4f}".format(r2))  
print ("MAE value: {:.4f}".format(mae))
```

```
In [ ]: from sklearn.neighbors import KNeighborsRegressor  
from sklearn.metrics import mean_absolute_error, r2_score  
  
knn = KNeighborsRegressor()  
knn.fit(X_train_scaled, y_train)  
knn_pred = knn.predict(X_test_scaled)  
mae = mean_absolute_error(y_test, knn_pred)  
r2 = r2_score(y_test, knn_pred)  
  
print("K_Neighbours Regressor")  
print ("R2 Score value: {:.4f}".format(r2))  
print ("MAE value: {:.4f}".format(mae))
```

```
In [ ]: param_grid = [{'bootstrap': [True, False],  
 'max_depth': [10, 20, 30, 40, 50, 60, 70, 80, 90, 100, 110, 120],  
 'max_features': ['sqrt', 'log2', None],  
 'min_samples_leaf': [1, 3, 4],  
 'min_samples_split': [2, 6, 10],  
 'n_estimators': [5, 20, 50, 100]]  
  
f_reg = RandomForestRegressor()  
Random_rf = RandomizedSearchCV(f_reg, param_grid, cv = 10, verbose=2, n_jobs = -1)  
Random_rf.fit(X_train_scaled, y_train)
```

```
In [ ]: best_random_grid=Random_rf.best_estimator_  
  
bestrf_pred = best_random_grid.predict(X_test_scaled)  
mae = mean_absolute_error(y_test, bestrf_pred)  
r2 = r2_score(y_test, bestrf_pred)  
  
print("Random Forest Tuned")  
print ("R2 Score value: {:.4f}".format(r2))  
print ("MAE value: {:.4f}".format(mae))
```

```
In [ ]: from sklearn.exceptions import FitFailedWarning, ConvergenceWarning  
import warnings  
warnings.filterwarnings(action='ignore')  
ridge_params = [{'alpha': (np.logspace(-11, 11, 100)),  
 'solver': ['auto', 'svd', 'cholesky', 'lsqr', 'sparse_cg', 'sag', 'saga', 'lbfgs']}]  
  
ridge = Ridge()  
random_ridge = RandomizedSearchCV(ridge, ridge_params, cv=10, n_jobs=-1)  
random_ridge.fit(X_train_scaled, y_train)
```

```
In [ ]: best_ridge = random_ridge.best_estimator_  
  
rbest_pred = best_ridge.predict(X_test_scaled)  
mae = mean_absolute_error(y_test, rbest_pred)  
r2 = r2_score(y_test, rbest_pred)  
  
print("Random Forest Tuned")  
print ("R2 Score value: {:.4f}".format(r2))  
print ("MAE value: {:.4f}".format(mae))
```

```
In [ ]: feature_importances = Random_rf.best_estimator_.feature_importances_  
importance_df = pd.DataFrame({  
 'feature': X_train.columns,  
 'importance': feature_importances  
}).sort_values('importance', ascending=False)  
importance_df
```

```
In [ ]: plt.figure(figsize=(12,6))  
sns.set_style('ticks')  
ax = sns.barplot(data=importance_df, x='importance', y='feature', ec = 'black')  
ax.set_title('Top 10 Important Features', weights='bold', fontsize = 15)  
ax.set_xlabel('Feature Importance %', weight='bold')  
ax.set_ylabel('Features', weights='bold')
```

```
In [ ]: Xtrain_new = X_train.drop(['Rain', 'Region', 'RH'], axis=1)  
Xtest_new = X_test.drop(['Rain', 'Region', 'RH'], axis=1)
```

```
In [ ]: Xtrain_new.columns
```

```
In [ ]: Xtrain_new_scaled, Xtest_new_scaled = scaler_standard(Xtrain_new, Xtest_new)
```

```
In [ ]: best_random_grid.fit(Xtrain_new_scaled, y_train)
best_rf_pred = best_random_grid.predict(Xtest_new_scaled)
mae = mean_absolute_error(y_test, best_rf_pred)
r2 = r2_score(y_test, best_rf_pred)

...
print("Random Forest Score: {}".format(r2))
```

```
print("Random Forest Score: {}".format(r2))
print("R2 Score value: {:.4f}".format(r2))
print("MAE value: {:.4f}".format(mae))
```

```
In [ ]: import bz2, pickle
file = bz2.BZ2File('regression.pkl', 'wb')
pickle.dump(best_random_grid, file)
file.close()
```

```
In [ ]: from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report
from sklearn.metrics import accuracy_score
from sklearn.metrics import ConfusionMatrixDisplay
from sklearn.neighbors import KNeighborsClassifier
from xgboost import XGBClassifier
```

```
In [ ]: X = df.drop('Classes', axis=1)
ys = df['Classes']
```

```
In [ ]: # separate dataset into train and test
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=36)
X_train.shape, X_test.shape
```

```
In [ ]: #Using Pearson Correlation
plt.figure(figsize=(12,10))
cor = X_train.corr()
sns.heatmap(cor, annot=True)
plt.show()
```

```
In [ ]: corr_features = correlation(X_train, 0.75)
```

```
In [ ]: corr_features
```

```
In [ ]: X_train.drop(corr_features, axis=1, inplace=True)
X_test.drop(corr_features, axis=1, inplace=True)
X_train.shape, X_test.shape
```

```
In [ ]: X_train_scaled, X_test_scaled = scaler_standard(X_train, X_test)
```

```
In [ ]: # Import Logistic Regression to Train from SKLearn
logreg = LogisticRegression(max_iter=200)
logreg.fit(X_train_scaled, y_train)
logreg_pred = logreg.predict(X_test_scaled)
score = accuracy_score(y_test, logreg_pred)
cr = classification_report(y_test, logreg_pred)

print("Logistic Regression")
print("Accuracy Score value: {:.4f}".format(score))
print(cr)
```

```
In [ ]: logreg_cm = ConfusionMatrixDisplay.from_estimator(logreg, X_test_scaled, y_test)
```

```
In [ ]: dtc = DecisionTreeClassifier()
dtc.fit(X_train_scaled, y_train)
dtc_pred = dtc.predict(X_test_scaled)
score = accuracy_score(y_test, dtc_pred)
cr = classification_report(y_test, dtc_pred)

print("Decision Tree")
print("Accuracy Score value: {:.4f}".format(score))
print(cr)
```

```
In [ ]: dt_cm = ConfusionMatrixDisplay.from_estimator(dtc, X_test_scaled, y_test)
```

```
In [ ]: rfc = RandomForestClassifier()
rfc.fit(X_train_scaled, y_train)
rfc_pred = rfc.predict(X_test_scaled)
score = accuracy_score(y_test, rfc_pred)
cr = classification_report(y_test, rfc_pred)

print("Random Forest")
print("Accuracy Score value: {:.4f}".format(score))
print(cr)
```

```
In [ ]: rf_cm = ConfusionMatrixDisplay.from_estimator(rfc, X_test_scaled, y_test)
```

```
In [ ]: # KNeighborsClassifier to Train from SKLearn
knnc = KNeighborsClassifier()
knnc.fit(X_train_scaled, y_train)
knn_pred = knnc.predict(X_test_scaled)
score = accuracy_score(y_test, knn_pred)
cr = classification_report(y_test, knn_pred)

print("KNeighbors Classifier")
print("Accuracy Score value: {:.4f}".format(score))
print(cr)
```

```
In [ ]: knn_cm = ConfusionMatrixDisplay.from_estimator(knnc, X_test_scaled, y_test)
```

```
In [ ]: # Import XGBClassifier to Train from SKLearn
xgb = XGBClassifier()
xgb.fit(X_train_scaled, y_train)
xgb_pred = xgb.predict(X_test_scaled)
score = accuracy_score(y_test, xgb_pred)
cr = classification_report(y_test, xgb_pred)

print("XGBoost Classifier")
print("Accuracy Score value: {:.4f}".format(score))
print(cr)
```

```
In [ ]: xgb_cm = ConfusionMatrixDisplay.from_estimator(xgb, X_test_scaled, y_test)
```

```
In [ ]: params=[
    "learning_rate" : (np.linspace(0,10, 100)) ,
    "max_depth" : (np.linspace(1,50, 25,dtype=int)),
    "min_child_weight" : [1, 3, 5, 7],
    "gamma" : [0.0, 0.1, 0.2 , 0.3, 0.4],
    "colsample_bytree" : [0.3, 0.4, 0.5 , 0.7])
Random_xgb = RandomizedSearchCV(xgb, params, cv = 10,n_jobs = -1)
Random_xgb.fit(X_train_scaled, y_train).best_estimator_

In [ ]: best_xgb = Random_xgb.best_estimator_
best_xgb.score(X_test_scaled,y_test)
best_xgb.score(X_test_scaled,y_test)
```

```
best_xgb_pred = best_xgb.predict(X_test_scaled)
cr = classification_report(y_test,bestxgb_pred)
print("FINAL XGB")
print ("Accuracy Score value: {:.4f}".format(score))
print (cr)
```

```
In [ ]: params = {
    "n_estimators" : [90,100,115,130],
    "criterion": ['gini', 'entropy'],
    "max_depth" : range(2,20,1),
    "min_samples_leaf" : range(1,10,1),
    "min_samples_split" : range(2,10,1),
    "max_features" : ['auto','log2']
}
random_rf = RandomizedSearchCV(rfc, params, cv = 10,n_jobs = -1)
random_rf.fit(X_train_scaled, y_train).best_estimator_
```

```
In [ ]: best_rf = random_rf.best_estimator_
best_rf.score(X_test_scaled,y_test)
bestrf_pred = best_rf.predict(X_test_scaled)
cr = classification_report(y_test,bestrf_pred)
print("FINAL Random Forest")
print ("Accuracy Score value: {:.4f}".format(score))
print (cr)
```

```
In [ ]: from sklearn.model_selection import StratifiedKFold, cross_val_score
skfold = StratifiedKFold(n_splits= 10,shuffle= True,random_state= 36)
```

```
In [ ]: cv_xgb = cross_val_score(best_xgb,X,y,cv=skfold,scoring='accuracy').mean()
print('CV Score XGB Tuned {:.4f}'.format(cv_xgb))
```

```
In [ ]: cv_rfcross_val_score(rfc,X,y,cv=skfold,scoring="accuracy").mean()
print('CV Score Random Forest {:.4f}'.format(cv_rf))
```

```
In [ ]: cv_lgscross_val_score(logreg,X,y,cv=skfold,scoring="accuracy").mean()
print('CV Score Logistic Regression {:.4f}'.format(cv_lg))
```

```
In [ ]: cv_dt = cross_val_score(dtc,X,y,cv= skfold,scoring="accuracy").mean()
print('CV Score Decision Tree {:.4f}'.format(cv_dt))
```

```
In [ ]: cv_knn = cross_val_score(knnc,X,y,cv=skfold,scoring="accuracy").mean()
print('CV Score KNN Classifier {:.4f}'.format(cv_knn))
```

```
In [ ]: feature_importances = Random_xgb.best_estimator_.feature_importances_
importance_df = pd.DataFrame({
    'feature': X_train.columns,
    'importance': feature_importances
}).sort_values('importance', ascending=False)
importance_df
```

```
In [ ]: plt.figure(figsize=(12,6))
sns.set_style('ticks')
ax = sns.barplot(data=importance_df, xs='importance', ys='feature',ec = 'black')
ax.set_title('Top 10 Important Features', weights='bold',fontsize = 15)
ax.set_xlabel('Feature Importance %',weights='bold')
ax.set_ylabel('Features',weights='bold')
```

```
In [ ]: Xtrain_new = X_train.drop(['Rain', 'Region', 'RH'], axis=1)
Xtest_new = X_test.drop(['Rain', 'Region', 'RH'], axis=1)
```

```
In [ ]: Xtrain_new_scaled, Xtest_new_scaled = scaler_standard(Xtrain_new, Xtest_new)
```

```
In [ ]: xgb_model = Random_xgb.fit(Xtrain_new_scaled, y_train).best_estimator_
xgb_model.score(Xtest_new_scaled, y_test)
xgb_model_pred = xgb_model.predict(Xtest_new_scaled)
cr = classification_report(y_test, xgb_model_pred)
print("Final Model XGB")
print ("Accuracy Score value: {:.4f}".format(score))
print (cr)
```

```
In [ ]: #Compressing the pickle file which has 5 features to build Web Application
import bz2,pickle
file = bz2.BZ2File('classification.pkl','wb')
pickle.dump(xgb_model,file)
file.close()
```

```
In [ ]: final_df = df.drop(['BUI','DC','Rain', 'Region','RH'],axis=1)
```

```
from pymongo import MongoClient
# Establish a connection to a MongoDB Atlas Cluster with Secured Authentication using User Name and Password of the Database
client = MongoClient("mongodb+srv://mongodb:12345@cluster0.bisvs.mongodb.net/?retryWrites=true&maxretry=5")
# Create Database and specify name of database
db = client.get_database('FireDataML')
# Create a collection
records = db.ml_task

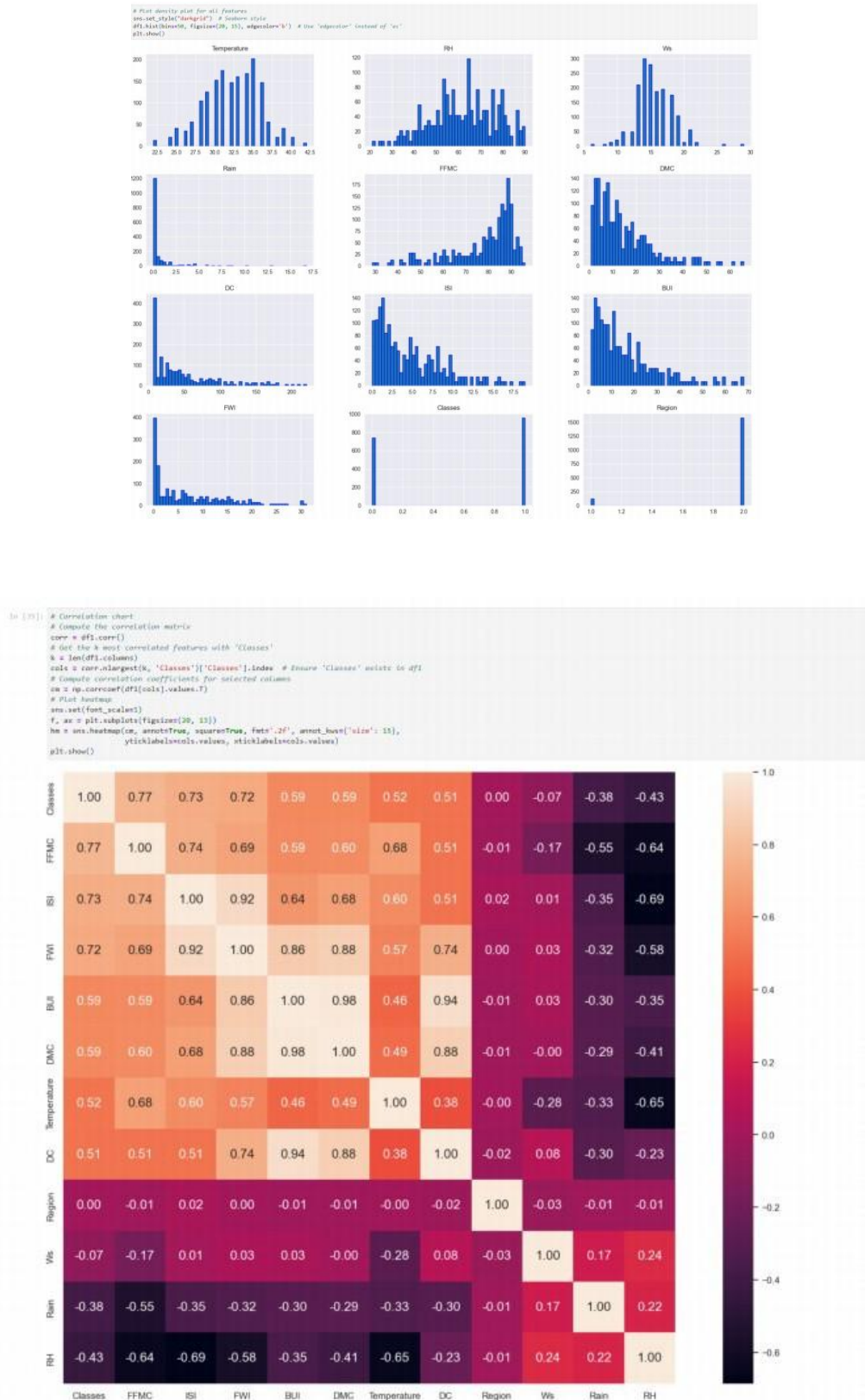
data = final_df.to_dict(orient = 'records')
db.ml_task.insert_many(data)
```

```
In [ ]: final_df
```

```
In [ ]:
```


APPENDIX B - OUTPUT

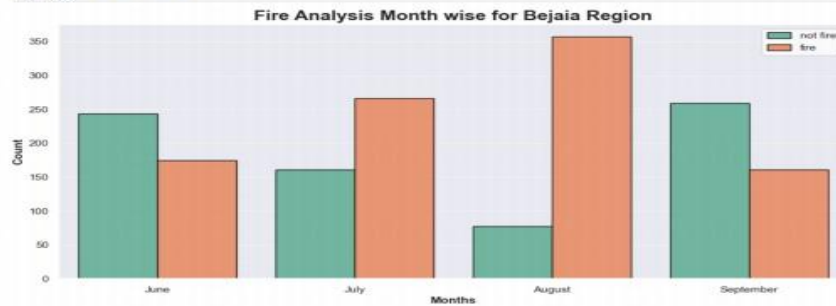
1. EDA:




```

24 [107]: dfTempo[df['Region']=='1']
plt.subplots(figsize=(15,8))
sns.set_style('whitegrid')
sns.countplot(x='month',hue='Classes',data=df,ec='black',palette='Set2')
plt.title('Fire Analysis Month wise for Bejaia Region', fontsize=18, weights='bold')
plt.xlabel('Count', weight='bold')
plt.ylabel('Months', weight='bold')
plt.legend(loc='upper right')
plt.xticks(rotation=0, ['June','July', 'August', 'September'])
plt.grid(alpha = 0.5,axis = 'x')
plt.show()

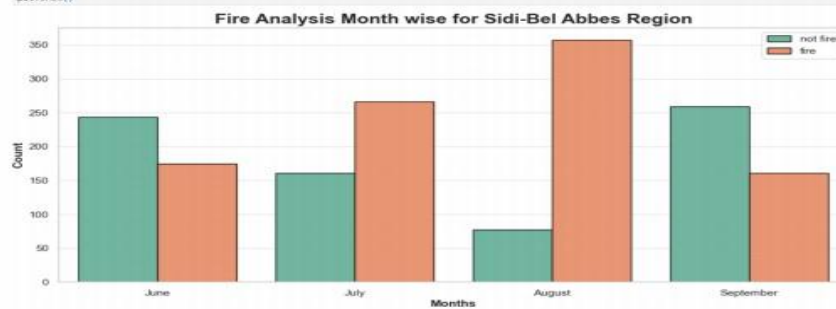
```



```

24 [108]: dfTempo[df['Region']=='2']
plt.subplots(figsize=(15,8))
sns.set_style('whitegrid')
sns.countplot(x='month',hue='Classes',data=df,ec='black',palette='Set2')
plt.title('Fire Analysis Month wise for Sidi-Bel Abbes Region', fontsize=18, weights='bold')
plt.xlabel('Count', weight='bold')
plt.ylabel('Months', weight='bold')
plt.legend(loc='upper right')
plt.xticks(rotation=0, ['June','July', 'August', 'September'])
plt.grid(alpha = 0.5,axis = 'x')
plt.show()

```



2. MODEL:

```

In [8]: X_train.corr()
Out[8]:

```

	Temperature	RH	Ws	Rain	FFMC	DMC	DC	ISI	BUI	Classes	Region
Temperature	1.000000	-0.653968	-0.285353	-0.357845	0.688107	0.499631	0.384916	0.603913	0.471025	0.511431	-0.017688
RH	-0.653968	1.000000	0.243605	0.239389	-0.639192	-0.414589	-0.227891	-0.690052	-0.357199	-0.431132	-0.003121
Ws	-0.285353	0.243605	1.000000	0.193517	-0.175270	-0.010054	0.071068	0.008714	0.023502	-0.066734	-0.021052
Rain	-0.357845	0.239389	0.193517	1.000000	-0.567232	-0.303434	-0.305760	-0.355700	-0.313004	-0.387993	-0.018722
FFMC	0.688107	-0.639192	-0.175270	-0.567232	1.000000	0.610983	0.511006	0.736913	0.598507	0.771430	-0.024505
DMC	0.499631	-0.414589	-0.010054	-0.303434	0.610983	1.000000	0.870441	0.677056	0.981831	0.590276	-0.051973
DC	0.384916	-0.227891	0.071068	-0.305760	0.511006	0.870441	1.000000	0.498593	0.941985	0.511050	-0.039774
ISI	0.603913	-0.690052	0.008714	-0.355700	0.736913	0.677056	0.498593	1.000000	0.640013	0.733030	-0.001334
BUI	0.471025	-0.357199	0.023502	-0.313004	0.598507	0.981831	0.941985	0.640013	1.000000	0.592555	-0.046902
Classes	0.511431	-0.431132	-0.066734	-0.387993	0.771430	0.590276	0.511050	0.733030	0.592555	1.000000	-0.007787
Region	-0.017688	-0.003121	-0.021052	-0.018722	-0.024505	-0.051973	-0.039774	0.001334	-0.046902	-0.007787	1.000000

```

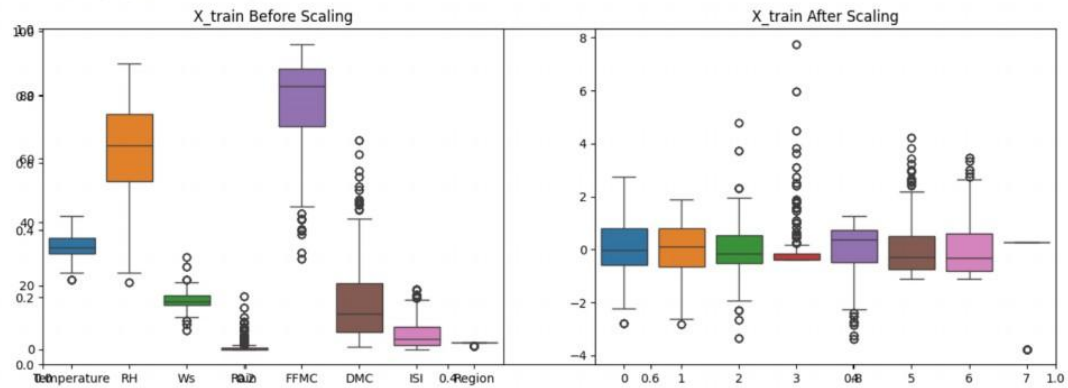
In [9]: #Using Pearson Correlation
plt.figure(figsize=(12,10))
cor = X_train.corr()
sns.heatmap(cor, annot=True, cmap=plt.cm.YlOrRd)
plt.show()

```



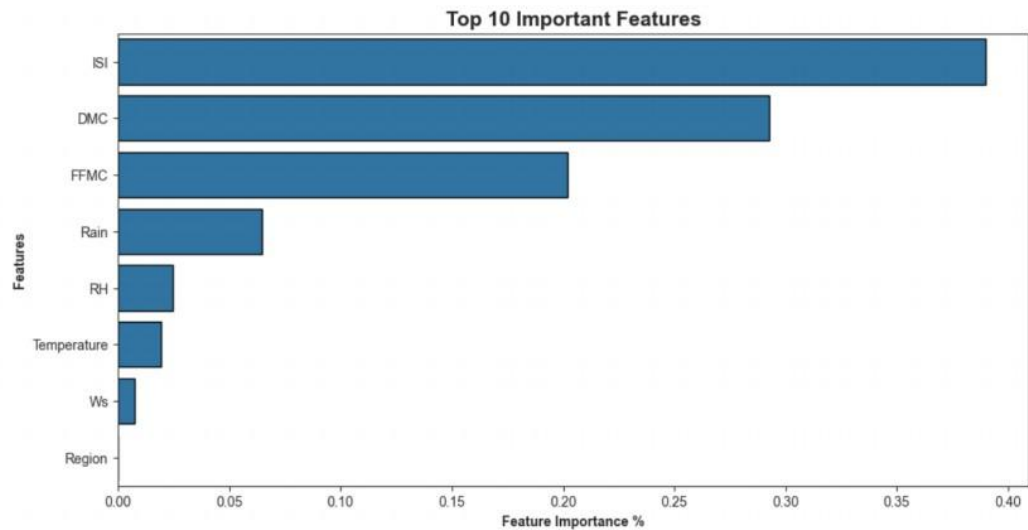
```
In [16]: plt.subplots(figsize=(15, 5))
plt.subplot(1, 2, 1)
sns.boxplot(data=X_train)
plt.title('X_train Before Scaling')
plt.subplot(1, 2, 2)
sns.boxplot(data=X_train_scaled)
plt.title('X_train After Scaling')
```

```
Out[16]: Text(0.5, 1.0, 'X_train After Scaling')
```



```
In [28]: plt.figure(figsize=(12,6))
sns.set_style('ticks')
ax = sns.barplot(data=importance_df, x='importance', y='feature', ec='black')
ax.set_title('Top 10 Important Features', weight='bold', fontsize=15)
ax.set_xlabel('Feature Importance %', weight='bold')
ax.set_ylabel('Features', weight='bold')
```

```
Out[28]: Text(0, 0.5, 'Features')
```



```
In [37]: #Using Pearson Correlation
plt.figure(figsize=(12,10))
cor = X_train.corr()
sns.heatmap(cor, annot=True)
plt.show()
```



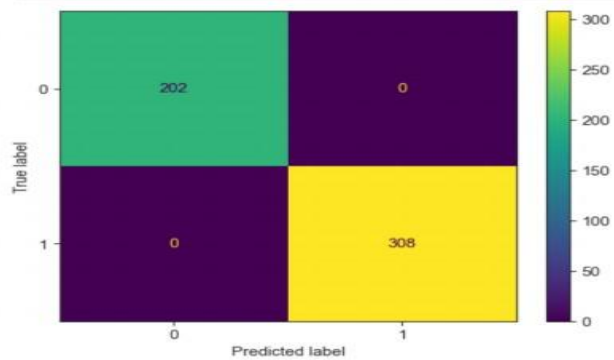
```
In [43]: loreg_cm = ConfusionMatrixDisplay.from_estimator(loreg, X_test_scaled, y_test)
```



```
In [43]: loreg_cm = ConfusionMatrixDisplay.from_estimator(loreg, X_test_scaled, y_test)
```



```
In [51]: xgb_cm = ConfusionMatrixDisplay.from_estimator(xgb, X_test_scaled, y_test)
```



```
In [52]: params={
    "learning_rate" : (np.linspace(0.10, 100)) ,
    "max_depth" : (np.linspace(1,50, 25,dtype=int)),
    "min_child_weight" : [1, 3, 5, 7],
    "gamma" : [0.0, 0.1, 0.2 , 0.3, 0.4],
    "colsample_bytree" : [0.3, 0.4, 0.5 , 0.7]}
Random_xgb = RandomizedSearchCV(xgb, params, cv = 10,n_jobs = -1)
Random_xgb.fit(X_train_scaled, y_train).best_estimator_
```

```
Out[52]: XGBClassifier
XGBClassifier(base_score=None, booster=None, callbacks=None,
    colsample_bylevel=None, colsample_bynode=None,
    colsample_bytree=0.7, device=None, early_stopping_rounds=None,
    enable_categorical=False, eval_metric=None, feature_types=None,
    gamma=0.4, grow_policy=None, importance_type=None,
    interaction_constraints=None,
    learning_rate=np.float64(0.8080808080808081), max_bin=None,
    max_cat_threshold=None, max_cat_to_onehot=None,
    max_delta_step=None, max_depth=np.int64(33), max_leaves=None,
```

```
In [69]: final_df
```

```
Out[69]:
```

	Temperature	Ws	FFMC	DMC	ISI	FWI	Classes
0	26	22	47.1	2.5	0.3	0.1	0
1	27	16	64.8	3.0	1.2	0.5	0
2	26	19	84.0	13.8	4.8	7.1	1
3	31	17	79.1	7.0	2.4	2.3	0
4	32	17	85.9	11.2	5.6	7.5	1
...	...	...	...	...	...	...	...
1695	30	14	85.4	16.0	4.5	6.5	1
1696	28	15	41.1	6.5	0.1	0.0	0
1697	27	29	45.9	3.5	0.4	0.2	0
1698	24	18	79.7	4.3	1.7	0.7	0
1699	24	15	67.3	3.8	1.2	0.5	0

1700 rows × 7 columns

